Optimizing Graph I/O for In-memory Computations

Ali Brooks and George M. Slota
Rensselaer Polytechnic Institute
{brooka9, slotag}@rpi.edu

***Abstract*—Graph I/O and preprocessing is a critical part of the approach with which we analyze all graph datasets. For many graph analytics, simply loading the graph from disk into memory can constitute the bulk of end-to-end times. Despite this, preprocessing and I/O-related costs are often ignored within publicly-available graph processing frameworks and analytic codes. In this work, we present highly parallelizable routines for the preprocessing, reading, writing, and in-memory construction of graph representations. We specifically consider the on-disk binary edge list format due to its ease of preprocessing and reading in parallel, alongside a standard CSR in-memory graph representation for shared and distributed memory under vertex and edge-based distributions. Our approach enables the reading and in-memory construction of the largest publicly available graph dataset, the 128 billion edge 2012 Web Data Commons crawl, within 40 seconds across 512 GPUs and 130 seconds across 512 CPU MPI ranks, significantly outperforming other methods.**

***Index Terms*—Graph Analytics, Graph I/O, Graph Processing**

I. Introduction

At the beginning of every graph-based computation, there is a critical step regarding how large graph representations are taken from disk, processed, and represented in memory. When dealing with small graphs, this processing time is often trivial. However, as graphs scale up towards billions of edges and beyond, this processing step can take a significant portion of the overall time of what might otherwise be a relatively quick computation. With the exception of out-of-core computation models [1, 2, 3], this preprocessing and I/O phase is often unmentioned in the literature, where the timing generally starts at the bespoke algorithm under consideration. This can lead to an incomplete picture regarding performance and benchmarking, where in practice any given implementation speedup is effectively made irrelevant by slow I/O and preprocessing.

This omission of I/O costs in the academic literature creates a significant gap between theoretical algorithmic performance and real-world application efficiency. By focusing exclusively on computation, many studies present an idealized scenario that does not account for the data-movement challenges inherent in large-scale analytics. Consequently, practitioners attempting to leverage recent research may find that reported speed-ups are unattainable in production environments, as the cost of loading data from storage or converting to an implementation-specific in-memory representation dwarfs any computational gains. This disconnect not only hinders the practical adoption of novel algorithms, but it also makes it difficult to perform fair and holistic comparisons between different graph processing frameworks.

A. Our Contributions

To address this gap, we present a study of the graph I/O and preprocessing bottleneck. Our primary contribution is the design, implementation, and evaluation of highly parallel algorithms that read a graph from disk and convert it to the appropriate in-memory representation. Our methods leverage single node CPU through multiple-node distributed GPU processing and communication backends, greatly accelerating the graph loading step relative to other work. We demonstrate that our approaches scale to 512 CPU/GPU MPI ranks, processing the largest publicly available 128 billion edge 2012 Web Data Commons graph dataset in under 40 seconds on 512 GPUs, from disk to a full CSR graph data structure. We additionally perform an experimental study, comparing I/O mechanisms in the popular graph processing libraries Ligra [4], Galois (Gluon) [5, 6], CombBLAS [7], and CuGraph [8]. We aim to quantify the I/O problem and present a viable solution that gives high-performance, low storage costs, and is generalizable across multiple graph computational contexts.

Specifically, we consider the graph I/O and preprocessing under the 4 distinct computational contexts of shared memory CPU, shared memory GPU, distributed memory CPU, and distributed GPU. We define I/O and preprocessing as the transfer of a graph stored on-disk into a standard in-memory serial or distributed CSR graph or adjacency-like format. Our approaches utilize a binary edge list format for storage on disk, and our presented algorithms demonstrate how to efficiently construct the CSR format quickly and in parallel under the aforementioned computational contexts. We compare against several well-established formats represented by the state-of-the-art graph processing frameworks above. These formats include those that directly use an adjacency-list or CSR (binary or text) storage format and those that use an edge list (e.g., matrix market) text format. We consider three primary metrics for evaluation: on-disk storage cost, preprocessing and I/O time, and all of the in-memory processing times.

B. Our Conclusion: The Case for Binary Edge List

Our primary conclusions can be stated simply as: The binary edge list format is one of the most space-efficient non-compressed storage formats on-disk, particularly for undirected graphs. I/O for this format is extremely fast and highly parallelizable in both shared and distributed memory. In situ graph construction costs can be higher relative to direct CSR or adjacency formats in shared memory, but this cost is easily justified by the more expensive offline preprocessing, storage, and distribution costs for the other formats.

II. Background

A. Basic Graph Representations

We will define a graph formally as the tuple $G = \{V, E\}$, where V is a set of vertices and E is a set of edges. An edge $e \in E$ can be represented as an unordered pair of vertices $\{u, v\} : u, v \in V$ for an undirected graph, or an ordered pair in a directed graph. A major focus of this work is specifically on the edge list E, which is used as the de facto graph storage for the most popular graph data repositories. Many graph datasets also consider edge weights, represented with weight w as part of edge tuple $\{u, v, w\}$. For simplicity, we will be considering unweighted graphs in this work, though adding weights is a trivial modification to most of the algorithms we discuss. For static in-memory computation, most graph processing frameworks utilize a compressed sparse row (CSR) format instead of an edge list, due to improved cache efficiency during computation and lower memory requirements for directed graphs. A CSR format utilizes two arrays, which we will term as $Adjacencies$ and $Offsets$. Assuming all $v \in V$ have unique numeric identifiers in $[0, \ldots, (|V| - 1)]$, $Offsets[v]$ represents a starting index in $Adjacencies$ for the opposite endpoints of all edges incident on v.

B. On-disk Graph Formats

We consider for discussion two primary graph storage formats: edge lists and adjacency lists. The edge list format stores a graph simply as a list of edge tuples defined as the $\{u, v\}$ pairs. Adjacency list formats store neighborhood lists for each vertex in order, with a direct CSR storage format being considered in this work as an adjacency list variant. All of these formats can be stored on disk as text or binary, with adjacency list formats also commonly used for compressed binary representations (though compressed formats are beyond the scope of this current work).

There currently exists about a half dozen well-established graph data repositories, with the SuiteSparse Matrix Collection [9], Stanford Large Network Dataset Collection [10], and the KONECT Project [11] perhaps being the most well known. These repositories use a text-based edge list format (e.g., matrix market) as the default storage format for their hosted graph datasets. Likewise, there is a wide breadth of existing graph processing frameworks (e.g., [4, 5, 7, 8], among many more). These frameworks generally either use a standard text edge list format or their own specialized format, which is often a binary adjacency-list variant. In our experimental study, we will analyze and discuss the benefits and drawbacks of the various possible formats, particularly under the considerations of: **1.)** Most graph datasets are available as text edge lists, and **2.)** On-disk graph data must be converted to some CSR or adjacency format in-memory for actual graph processing.

III. Methodology

In this section, we provide a description of our methodology for in-memory graph construction from an on-disk graph file. As noted, we specifically consider our on-disk representation as a binary edge list, constructed from a text edge list or equivalent representation. The overall process for disk→memory→computation is given in Algorithm 1. Initially, a downloaded file is converted to the edge list binary format offline. During the actual computational routine, that file is then ingested into memory, its edges distributed among computational ranks (if necessary), with the graph data structure created before computation can proceed. In the following subsections, we will describe our implementations of each of the functions given in Algorithm 1.

Algorithm 1 Data File to Graph Processing

1: $F_b \leftarrow$ Preprocess(F_t) ▷ Preprocessing of text to binary
2: **Begin Graph Program**
3: $E \leftarrow$ ReadBinaryEdgeList(F_b)
4: $E' \leftarrow$ DistributeEdges(E) ▷ If in distributed memory
5: $G \leftarrow$ ConstructGraph(E')
6: PerformGraphComputation(G)

A. Implementation Details

As noted previously, we consider both shared memory and distributed memory CPU and GPU computational contexts. We developed our I/O framework in C++ with CUDA for GPU kernels, utilizing OpenMP for shared-memory parallelization and MPI/NCCL for distributed message passing. For distributed CPU processing, we use a standard 1D distribution, where ranks own a disjoint subset of vertices and all edges immediately incident on those vertices. For distributed GPU processing, we use a 2D edge-based distribution [6, 7, 12], where disjoint subsets of vertices have shared ownership among some number of rank, with incident edges for those vertices arbitrarily distributed among those ranks. The above routines are all relatively consistent under the differing computational environments, with the primary difference being in how vertex and edge ownership is determined during the edge exchange phase. Our code is hosted in a publicly available GitHub repository[1].

B. Text to Binary Edge List Preprocessing

Given in Algorithm 2 is our processing methodology for converting an edge list from a text to a binary representation, considering large-scale graphs and a parallel filesystem. We describe the algorithm as running in shared memory, but the modifications from this baseline algorithm are trivial for distributed memory.

Initially, each thread determines an approximately equal portion of the input file for reading, with a read starting and ending offsets (r_s, r_e) calculated based on its unique ID. A thread first counts the number of edges allocated to them (m) in the file using looped calls to `memchr`($currentOffset$, `'\n'`, ...). These counts are reduced into a global count M, which is used to allocate the output file via `posix_fallocate`($2 \times M \times$ `sizeof(edge_type)`). Until the entire file is converted, each thread reads a portion of the input file into the local buffer, calculates an offset into the output file, and outputs its buffer. The `BUFFER_SIZE` can

[1] https://github.com/HPCGraphAnalysis/GraphIO

be tuned based on graph scale and memory capacity. For our experiments, we set it to be large enough to contain the entire read portion for each thread, to optimize I/O traffic on our test server, which had sufficient memory do this for all but our largest test case.

Algorithm 2 Edge Binary Preprocessing

```
1: procedure PREPROCESS(Text Edge List File F_t)
2:     Begin Parallel
3:     r_s, r_e ← GetMyReadOffsets(F_t)        ▷ Start/end file offsets
4:     m ← CountEdges(F_t, [t_s, t_e])
5:     M ← Reduce(m)                           ▷ Get global number of edges
6:     Single Thread: F_b ← AllocateOutputFile(M)
7:     for r_c = r_s to r_e by BUFFER_SIZE do
8:         t_buff ← ReadEdges(F, t_c, BUFFER_SIZE)
9:         t_o ← GetWriteOffset(O, r_c)
10:        F_b[t_o] ← WriteEdges(t_buff)
11:    End Parallel
```

Our preprocessing implementation is trivially parallelized into $O(1)$ parallel depth, with real performance mostly being a function of filesystem I/O bandwidth and parallel capability. Using a fixed buffer size, preprocessing can also be accomplished with $O(1)$ (and arbitrarily small, in practice) memory. Necessarily, total work of $O(M)$ is required, where M is the number of edges in the input edge list.

C. Reading Binary Edge List From Disk

Our approach for reading a binary edge list into memory follows a similar pattern as in Algorithm 2. We allocate an edge list in shared memory, with each thread reading a portion of the input file into this shared array. In distributed memory, the global binary edge list is portioned off between MPI ranks before being further subdivided among any shared memory threads. Each thread effectively reads $\frac{M}{n_{ranks} \times n_{threads}}$ edges. When considering graph processing on GPU(s), we first read from the file using CPU threads into a CPU buffer before asynchronously copying the edges into GPU memory.

Many graph frameworks also perform a bijective vertex ID relabeling step before finalizing the graph data structure. This might relabel vertices in a specified order to improve cache locality or load balance, sometimes given via an explicit graph partitioning-based mapping. We perform this step, if desired, once our edge list is fully in CPU or GPU memory. In distributed memory, this reordering is utilized to drive the subsequent edge exchange and distribution step that finalizes the owned edge lists for each MPI rank.

As is the case for preprocessing the edge list, reading an edge list from disk into memory can be trivially parallelized into $O(1)$ depth. As the entire edge list is assumed to reside in memory for subsequent graph construction, the memory cost of this step is $O(M)$. Likewise, the total work will be $O(M)$.

D. Distributed Edge Exchange

In distributed memory, it is generally impractical for a rank to parse the entire on-disk graph representation in order to read and retrieve the rank's owned edges. Distributed on-disk storage containing a specific edge→rank distribution is possible, but such representations cannot generalize to arbitrary rank counts or varying partitionings. Hence, a common part of distributed parallel graph I/O is an exchange of edges from a rank that reads a specific edge to the rank that will own that edge for the graph computation. Given in Algorithm 3 is the basic process for this exchange.

Algorithm 3 Distributed Memory Edge Exchange

```
1: procedure DISTRIBUTEEDGES(Edge List E)
2:     Counts_Send ← CountEdgesPerRank(E)
3:     Buff_Send ← ConstructSendBuffer(E, Counts_Send)
4:     Counts_Recv ← ExchangeSendCounts(Counts_Send)
5:     E' ← AllocateReceiveBuffer(Counts_Recv)
6:     E' ← ExchangeSendBuffers(E, Counts_Send, Counts_Recv)
```

After reading their portion of the global edge list, every rank determines ownership of each edge of their edges, counting the number of edges to send to every other rank. A send buffer is then constructed for an MPI-style all-to-all exchange of these edges. Both of these routines are straightforward to parallelize. Counts and edge placement in the send buffer require atomic synchronization, which can be made more efficient by doing thread-owned count arrays on CPU and thread, warp, and block level intermediate reductions on GPU.

The computational routines are parallelizable to $O(1)$ depth with $O(M)$ work and memory. The practical performance of this phase is going to be a function of the all-to-all exchange. While it requires only $O(M)$ communication volume, the number of messages scales as $O(p^2)$, where p is the number of ranks. We perform this 1D-style communication even when working with 2D distributions, as this one-off cost is not observed to be a bottleneck in practice at our largest scale.

E. Graph Construction

Converting an edge list into an adjacency-based format like CSR requires three primary phases, given in Algorithm 4. Initially, the edge list is parsed to determine the degrees of each vertex. For a CSR graph, prefix sums are then calculated on these degrees to construct the $Offsets$ array. After allocating the $Adjacencies$ array, a copy of the offsets array is then created, which is used to fill in adjacencies. When a given edge (u, v) is processed, $Adjacencies[Offsets[u]]$ is set to v while $Offsets[u]$ is atomically incremented. A final (optional) step is to sort the adjacencies of each vertex within the CSR. This sorting improves cache efficiency on CPU and coalescing of memory accesses on GPU.

Algorithm 4 Graph Construction using a CSR-like Format

```
1: procedure CONSTRUCTGRAPH(Edge List E')
2:     Offsets ← CountDegrees(E')
3:     Offsets ← PrefixSums(Offsets)            ▷ CSR Offsets array
4:     Offsets_copy ← Copy(Offsets)
5:     Adjacencies ← ConstructAdjList(E', Offsets_copy)
6:     Adjacencies ← SortAdjacencies(Adjacencies, Offsets)
7:     G ← {Adjacencies, Offsets}
```

Each of these phases is relatively straightforward to parallelize. Similar to the edge exchange routines, each update to $Offsets$ in our degree counting phase and $Offsets_{copy}$

in the adjacency list construction must be atomically incremented. We have not observed this as a major bottleneck for graphs with vertices of massive degree. At a memory cost much higher than needed for counts in the edge exchange phase, intermediate reductions at the thread (on CPU) and warp/block (on GPU) level could also be used to minimize atomics. In distributed memory, graph construction is going to be consistent with the approach in Algorithm 4, simply outputting a local graph for a given edge list owned by the rank. However, there might be some additional overheads or considerations, depending on the graph framework and communication substrate being utilized.

Overall, CountDegrees() and ConstructAdjList() can be parallelized into $O(1)$ depth procedures when assuming a CRCW-PRAM model. The prefix sums and sorting procedures would require depths of $O(\log(N))$ and $O(\log^2(d_{max}))$, respectively, where N is the number of vertices and d_{max} is the maximum local degree on a rank. Memory cost and work for graph construction is otherwise bounded by $O(N + M)$, assuming a work-efficient prefix sums algorithms is being utilized.

IV. Experimental Setup

We use three systems for our experiments, representing single node CPU, single node GPU, and multi-node CPU/GPU environments. Our CPU system is `bella`, which has 2× AMD EPYC 7742 64-core CPUs with 2 TB DRAM and 8×16TB SATA 6.0Gb/s 7200RPM drives in RAID 6 controlled via a LSI MegaRAID 9361-8i SAS 12Gb/s PCIe 3.0 8-Port Controller. Our GPU system is `zepy`, which has 2× Intel Xeon Gold 6258R 28-core CPUs with 512 GB DRAM and 4× NVIDIA 40 GB A100 GPUs and a single 14TB SATA 6.0Gb/s 7200RPM drive for bulk storage. Our distributed test system is `AiMOS`, which contains nodes with 2× IBM Power-9 CPUs and 512 GB DRAM, 6× NVIDIA 32 GB V100 GPUs, all connected via an EDR Infiniband network with a unified 11-petabyte IBM Spectrum Scale (GPFS) parallel filesystem with a listed maximum bandwidth of 80 GB/s. Sequential parallel read bandwidth was benchmarked on `zepy` and `bella` at 0.52 GB/s 3.9 GB/s, respectively, using the `fio` utility.

Name	Vertices	Edges	Description	Source
USA-road-d	24 M	53 M	Road Network	[13]
twitter-2010	41 M	1.5 B	Follower Network	[14]
com-friendster	65 M	1.8 B	Social Network	[15]
MOLIERE_2016	30 M	3.3 B	Hypothesis Network	[16]
web-ClueWeb09	1.7 B	7.9 B	Web Crawl	[17]
gsh-2015	1.0 B	34 B	Web Crawl	[18]
WDC12	3.6 B	128 B	Web Crawl	[19]

TABLE I
GRAPH INPUT DATASETS FOR EXPERIMENTS WITH NUMBER OF VERTICES AND EDGES AND THE ORIGINAL DATA SOURCE.

To evaluate performance across various graphs and graph structures and scales, we used the public and well-known datasets listed in Table I. These datasets span over a 2000× difference in scale, from tens of millions of edges to hundreds of billions of edges. During our experiments, we treat all edges as undirected and mirror (or 'symmetrize') them across all tests for ease of comparison across datasets and frameworks. We do not explicitly deduplicate multi-edges, remove self loops, or reorder vertex IDs. We fully clear filesystem I/O cache between each timing run on `zepy` and `bella`. As we were unable to do this on `AiMOS`, we report the timings with a fully preheated cache (by pre-reading the file) for consistency.

Framework	Context	Format	Par. I/O	Sorted	Citation
Ligra	Shared CPU	Text/Bin. adj.	N	N	[4]
CuGraph	Dist. GPU	Text edge	Y	Y	[8]
CombBLAS	Dist. CPU	Text edge	N	Y	[7]
Galois (Gluon)	All	Binary adj.	Y	Y	[5, 6]
This Work	All	Binary edge	Y	Y	[12, 20]

TABLE II
GRAPH FRAMEWORKS, COMPUTATIONAL CONTEXTS, GRAPH STORAGE FORMAT UTILIZED, WHETHER THREAD-BASED PARALLELIZATION OF FILE I/O IS SUPPORTED BEYOND MPI PARALLELIZATION, AND WHETHER THE ADJACENCIES IN THE CSR-LIKE GRAPH STRUCTURE ARE SORTED DURING GRAPH CONSTRUCTION OR FILE CONVERSION.

We selected 4 representative graph computational framework to cover the listed combinations of computational contexts for single node/distributed CPU/GPU processing and data formats, listed in Table II. These works represent the most popular shared memory CPU framework (Ligra), the most scalable distributed CPU and GPU general framework in the literature outside of our own work (Galois+Gluon), as well as the most popular actively maintained frameworks for distributed GPU (CuGraph) and CPU (CombBLAS) graph processing. However, we note that we have been unable to get CuGraph to run on our IBM-based `AiMOS` system, so we were only able to perform comparisons in the single node environment of `zepy`. We run CombBLAS, Galois-GPU, and our GPU methods under a 2D distribution. These frameworks also cover the graph storage formats of binary and text adjacency list and text edge list. We use the smallest types possible (32 vs 64-bit) for each graph and framework. We know of no other work that directly utilizes binary edge lists for I/O. For all experiments across all frameworks, we maximized thread-based parallelism when applicable.

V. Results

We perform two primary sets of experimental evaluation. First, we analyze the strong scalability of our methods up to 512 MPI ranks and GPUs (both across 86 nodes) on `AiMOS`. Subsequently, we perform comparisons of methods utilizing text edge list and binary adjacency list methods on 64 Ranks/GPUs (11 nodes of `AiMOS`) and on single node CPU (`bella`) and GPU (`zepy`) systems.

A. Strong Scaling

Given in Figure 1 is strong scaling results across all datasets from 1 to 512 CPU MPI ranks and 1 to 512 GPUs on AiMOS. For our 2D distribution on GPU, we run in a square grid or as close to a square grid as possible biased towards the row dimension (e.g., 64 total GPUs in a 8×8 grid and 512 GPUs in a 32×16 grid). We break execution time down into the three

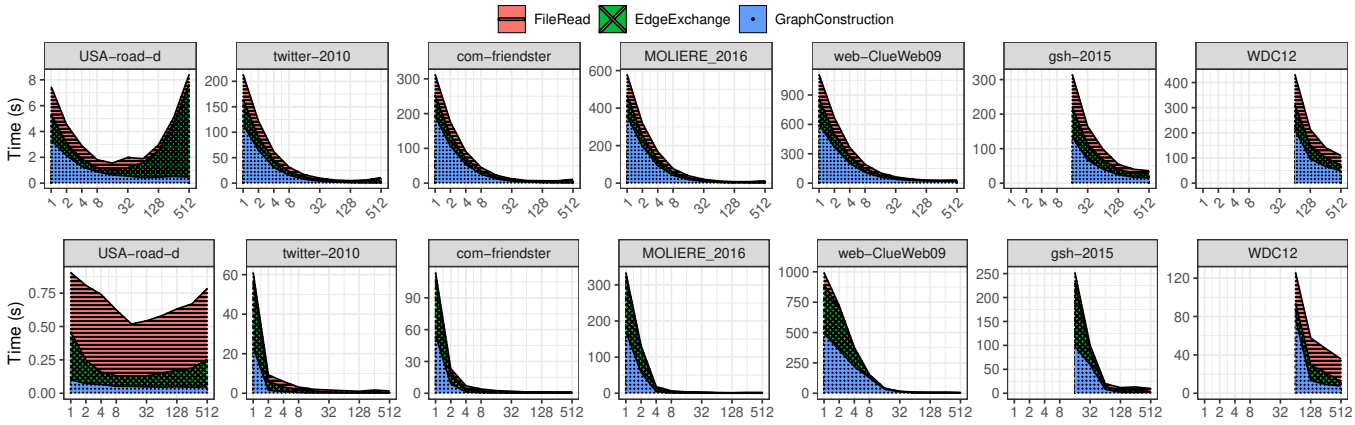


Fig. 1. We give strong scaling from 1 to 512 CPU MPI ranks (top) and GPUs (middle) across all test instances on AiMOS. We break total execution time into the three described phases of graph reading (FileRead), edge exchange (EdgeExchange), and graph construction (GraphConstruction).

described constituent phases for distributed memory, including edge list read, edge exchange, and graph construction. We note that Graph Construction tends to dominate overall execution time on CPU, while edge exchange is proportionally slowest on GPU at low ranks and file reading is slowest at higher ranks. For graph construction, each rank must traverse through their entire edge list twice during degree counting and adjacency list construction; on CPU, the lower memory bandwidth and thread counts per rank imply lower potential on-node speedup during this phase compared to GPU. On GPU, the increase in file reading time relative to edge exchange can be described as a function of effective filesystem vs network bandwidth relative to number of ranks. Available filesystem bandwidth is listed at 80 GB/s in total while network bandwidth is listed at 200 Gb/s per compute node, so filesystem bandwidth becomes saturated much quicker as we scale out. While this effect is consistent on CPU as well, the higher graph construction costs still dominate overall end-to-end times. On our smallest test input (USA-road-d), we note that the edge exchange on CPU dominates at the largest rank counts due to latency effects. On GPU, this exchange is much faster, most likely due to the more optimized NCCL vs. Spectrum MPI libraries. For the largest input of WDC12, we can read, exchange, and construct the graph in about 130 seconds on CPU and in under 40 seconds on GPU.

B. Comparisons

Preprocessing and Storage Costs: Given in Figure 2 (top) is the total preprocessing times on `bella` for Galois, Ligra, and our own work across all datasets, except for gsh-2015 and WDC12. We use the `dist-graph-covert` utility within Galois with one rank per core, as this configuration offered the best average performance. Ligra failed to convert gsh-2015 due to memory errors. We note that our times are generally 1-3 orders-of-magnitude faster, due to our efficient parallel file reading and writing. If we were to use our methods to output an adjacency format instead of binary edge list, we expect our timings to double at most, given known relative graph construction times reported in e.g., Figures 1 and 3.

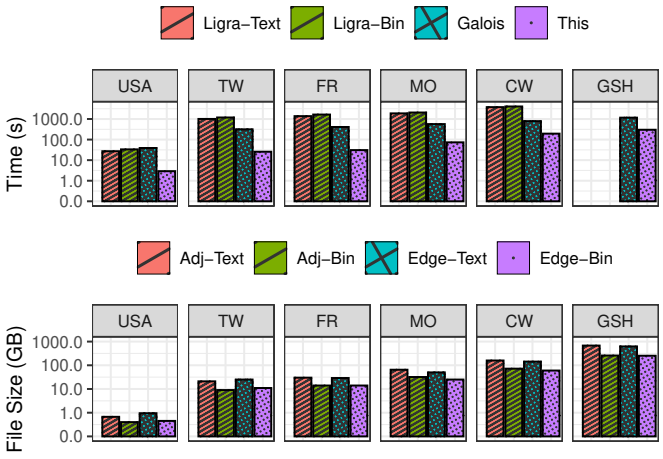


Fig. 2. Preprocessing time (top) of the various graph frameworks that use a non-text edge list format and storage cost (bottom) of the various formats.

Given in Figure 2 (bottom) is the storage cost comparison between binary and text edge lists and adjacency lists. Generally, for undirected graphs and symmetric matrices, binary edge list formats are going to require slightly less storage than binary adjacency lists, as the primary difference in storage costs is the offsets array needed for adjacency lists. For directed graphs and non-symmetric matrices, binary edge lists would be expected to require up to double the storage.

Single and Multi-Node Timing Comparisons: In Figure 3 we give experimental timings for single node (top) and 64-rank/GPU (bottom) experiments across all inputs and frameworks. As before, we break down total execution time into file reading, edge exchange (when applicable) and graph construction. Missing columns indicate where a framework/graph combination failed, generally due to out-of-memory or similar errors. On average, our approach is considerably faster than all other methods, offering considerable average speedups. In particular, we note that many other methods have a relatively higher proportion of time for file reading, suggesting con-

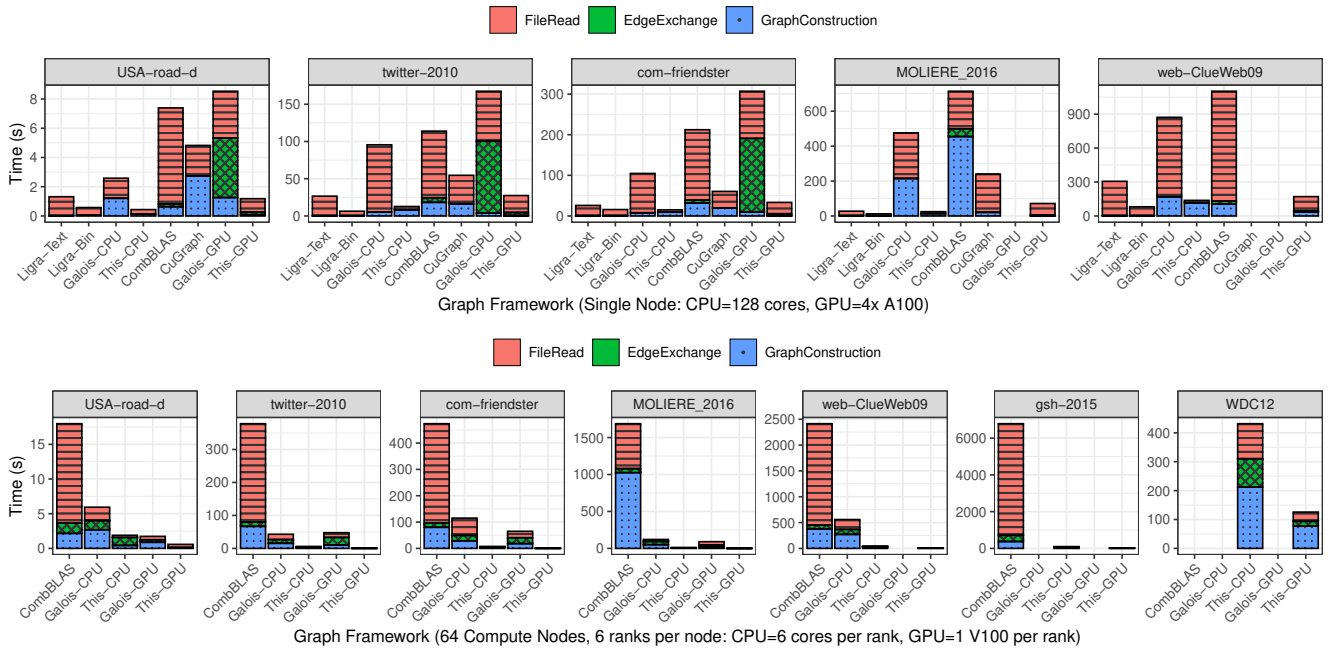


Fig. 3. Single (top) and multi-node (bottom) comparisons among all of the frameworks and tests graphs. We run on `bella` for single node CPU, `zepy` for single node GPU, and 64 ranks/GPUs of `AiMOS` for all multi-node experiments. For the distributed methods (CuGraph, Gluon-GPU, CombBLAS, This-GPU), we run 4 and 64 ranks with 2D 2×2 and 8×8 processor grids on `zepy` and `AiMOS`, respectively.

siderable room for graph frameworks to improve I/O routine efficiency in general.

End-to-end on Single Node: To highlight the differences in formats and frameworks discussed, we select com-friendster as a representative graph for comparison across all frameworks for single node end-to-end processing in Table III. The top rows of Table III were run on `bella` and the bottom rows on `zepy`. Essentially, Table III represents the total time cost to actually begin running some graph analytic algorithm on the dataset, assuming we already have com-friendster downloaded from e.g., SNAP or SuiteSparse in a common matrix market or edge list format. We note several takeaways.

		Timings (s)				
Framework	Format	PreProc	I/O	Graph	Total	Disk (GB)
CombBLAS	Edge-Text	**0.0**	337	259	596	29.0
Ligra	Adj-Text	1378	26.0	0.02	1404	29.6
Ligra	Adj-Bin	1644	15.7	0.09	1660	13.9
Galois-CPU	Adj-Bin	410	97	7.89	1414	13.9
This	Edge-Bin	21.5	**5.68**	9.30	36.5	**13.5**
CuGraph	Edge-Text	**0.0**	69.4	19.5	88.9	29.0
Galois-GPU	Adj-Bin	249	297	9.53	743	13.9
This	Edge-Bin	21.5	32.5	**0.96**	**55.0**	**13.5**

TABLE III
GRAPH FRAMEWORKS, THE EXPERIMENTED COMPUTATIONAL CONTEXT, AND THE GRAPH STORAGE FORMAT UTILIZED.

First, preprocessing for adjacency formats dominates the total end-to-end times, though this can be considered as a one-time-cost and there is considerable room for acceleration beyond the given timings for these specific frameworks. Secondly, the usage of an edge list format offers the very distinct tradeoff of lower to zero preprocessing costs at a cost of higher graph construction costs; however, this tradeoff is minimally impactful at the very large scale and with non-standard edge-based distributions, where a direct utilization of an adjacency format is not feasible. Lastly, we note the most important takeaway and the primary motivation of this work: **on average, graph I/O is not considered an important target for optimization**, despite the fact that for many of the frameworks listed when running many common graph analytics, the bulk of execution time is going to be everything necessary to construct the graph in memory before any analytic or graph algorithm can actually proceed.

VI. CONCLUSIONS

This paper describes a highly parallelizable and generalizable approach for preprocessing, reading, and constructing an in-memory graph representation from a graph dataset stored in a common format. We note that the primary motivation for this work is that graph I/O, as we term it, is often ignored within the graph analytics community, despite it often being one of the slowest phases in the processing and analysis of large graph datasets. Our experiments highlight this idea, while showing that our approach can fully read, load, and distribute the largest publicly available graph dataset across 512 GPUs in under 40 seconds. We believe our approach to be the fastest in practice for both CPU and GPU, while retaining optimal complexities for memory, work, and parallel depth.

VII. ACKNOWLEDGMENTS

This work is supported by the National Science Foundation under Grant No. 2047821. We also thank the HPEC reviewers for their helpful comments, inputs, and corrections.

References

[1] K. Vora, G. Xu, and R. Gupta, "Load the edges you need: A generic I/O optimization for disk-based graph processing," in *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, 2016, pp. 507–522.

[2] R. Wang, Y. Li, H. Xie, Y. Xu, and J. C. Lui, "GraphWalker: An I/O-Efficient and Resource-Friendly graph analytic system for fast and scalable random walks," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 559–571.

[3] D. Wen, L. Qin, Y. Zhang, X. Lin, and J. X. Yu, "I/o efficient core graph decomposition at web scale," in *2016 IEEE 32nd international conference on data engineering (ICDE)*. IEEE, 2016, pp. 133–144.

[4] J. Shun and G. E. Blelloch, "Ligra: a lightweight graph processing framework for shared memory," in *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, 2013, pp. 135–146.

[5] R. Dathathri, G. Gill, L. Hoang, H.-V. Dang, V. Jatala, V. K. Nandivada, M. Snir, and K. Pingali, "Gluon-Async: A Bulk-Asynchronous System for Distributed and Heterogeneous Graph Analytics," in *Proceedings of the 28th International Conference on Parallel Architectures and Compilation Techniques (PACT 2019)*, ser. PACT '19. IEEE, 2019.

[6] V. Jatala, R. Dathathri, G. Gill, L. Hoang, V. K. Nandivada, and K. Pingali, "A study of graph analytics for massive datasets on distributed multi-gpus," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2020, pp. 84–94.

[7] A. Azad, O. Selvitopi, M. T. Hussain, J. R. Gilbert, and A. Buluç, "Combinatorial blas 2.0: Scaling combinatorial algorithms on distributed-memory systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 989–1001, 2021.

[8] S. Kang, J. Nke, and B. Rees, "Analyzing multi-trillion edge graphs on large gpu clusters: A case study with pagerank," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2022, pp. 1–7.

[9] S. P. Kolodziej, M. Aznaveh, M. Bullock, J. David, T. A. Davis, M. Henderson, Y. Hu, and R. Sandstrom, "The suitesparse matrix collection website interface," *Journal of Open Source Software*, vol. 4, no. 35, p. 1244, 2019.

[10] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," http://snap.stanford.edu/data, Jun. 2014.

[11] J. Kunegis, "Konect: the koblenz network collection," in *Proceedings of the 22nd international conference on world wide web*, 2013, pp. 1343–1350.

[12] G. M. Slota and M. Mandulak, "Scaling distributed graph processing to hundreds of GPUs," in *International Conference on Parallel Processing (ICPP)*, 2025.

[13] R. A. Rossi and N. K. Ahmed, "The network data repository with interactive graph analytics and visualization," in *AAAI*, 2015. [Online]. Available: https://networkrepository.com

[14] H. Kwak, C. Lee, H. Park, and S. Moon, "What is Twitter, a social network or a news media?" in *Proceedings of the 19th international conference on World wide web*. ACM, 2010, pp. 591–600.

[15] A. Mislove, M. Marcon, K. P. Gummadi, P. Druschel, and B. Bhattacharjee, "Measurement and analysis of online social networks," in *IMC '07: Proceedings of the 7th ACM SIGCOMM conference on Internet measurement*. ACM, 2007, pp. 29–42.

[16] I. S. Justin Sybrandt, Michael Shtutman, "MOLIERE: automatic biomedical hypothesis generation system," in *KDD '17: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2017, pp. 1633–1642.

[17] C. L. Clarke, N. Craswell, and I. Soboroff, "Overview of the trec 2009 web track," DTIC Document, Tech. Rep., 2009.

[18] P. Boldi, A. Marino, M. Santini, and S. Vigna, "BUbiNG: Massive crawling for the masses," in *Proceedings of the Companion Publication of the 23rd International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee, 2014, pp. 227–228.

[19] R. Meusel, O. Lehmberg, C. Bizer, and S. Vigna, "Web data commons - hyperlink graphs," 2012.

[20] G. M. Slota, S. Rajamanickam, and K. Madduri, "A case study of complex graph analysis in distributed memory: Implementation and optimization," in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2016, pp. 293–302.